CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Chandubhai S Patel Institute of Technology

# Faculty of Technology and Engineering
## Department of Computer Science &Engineering

---

## Practical 11

| | | | | | |
|---|---|---|---|---|---|
| Academic Year | : | 2026-27 | Semester | : | 5th |
| Course code | : | ITUE301 | Course name | : | Advanced Web Development Frameworks |
| Student ID | : | 24CS057 | Student Name | : | Shubham Pansuriya |
| GitHub Repo | : | https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%2011 | | | |

### ❖ Problem Definition & Objectives

Course Outcome (CO/PO): CO5 / PO3, PO5 - To containerize the full-stack application using Docker and orchestrate services using Docker Compose.

Objective: To master container lifecycle management, multi-stage Docker builds, bridge network service resolution, and volume persistence in multi-tier web applications.

Problem Statement Requirements:

- Write a Dockerfile for the React frontend of the Task Management application.
- Write a Dockerfile for the Node/Express backend.
- Create a docker-compose.yml file that orchestrates all 3 services: frontend (5173), backend (5000), and MongoDB (27017).
- Configure correct port mappings and an internal Docker network so the services can communicate.
- Ensure the entire application starts with a single 'docker-compose up' command, with no manual intervention.

Supplementary Problems Completed:

- Supplementary 1: Added .dockerignore files to exclude node_modules and .env from the build context.
- Supplementary 2: Implemented a multi-stage Docker build for the React frontend, decoupling build dependencies from the lean production image.
- Supplementary 3: Configured a persistent named Docker volume (mongo-data) for MongoDB so task documents survive container restarts.

### ❖ Architecture & Docker Compose Orchestration Pipeline

```
docker-compose.yml
 ├── frontend (React 18 + Vite Preview, port 5173:5173)
 ├── backend (Node.js Express REST API, port 5000:5000)
```

```
└── mongodb (Official mongo:6.0 image, port 27017:27017)

Network: app-network (Custom Bridge Driver)

┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐
│ awdf-frontend    │ ──►  │ awdf-backend     │ ──►  │ awdf-mongodb     │
│ (172.20.0.4)     │      │ (172.20.0.3)     │      │ (172.20.0.2)     │
└──────────────────┘      └──────────────────┘      └──────────────────┘
        ▲                         ▲                         ▲
        │ :5173                   │ :5000                   │ :27017
┌──────────────────────────────────────────────────────────────────────┐
│                    Host Machine (Browser / Postman)                  │
└──────────────────────────────────────────────────────────────────────┘
                                                          │
                                                Persistent Volume
                                              [mongo-data:/data/db]
```

## ❖ Output Screenshots & Verification

### 1. Multi-Container Build & Startup (PowerShell Terminal – docker-compose up --build)

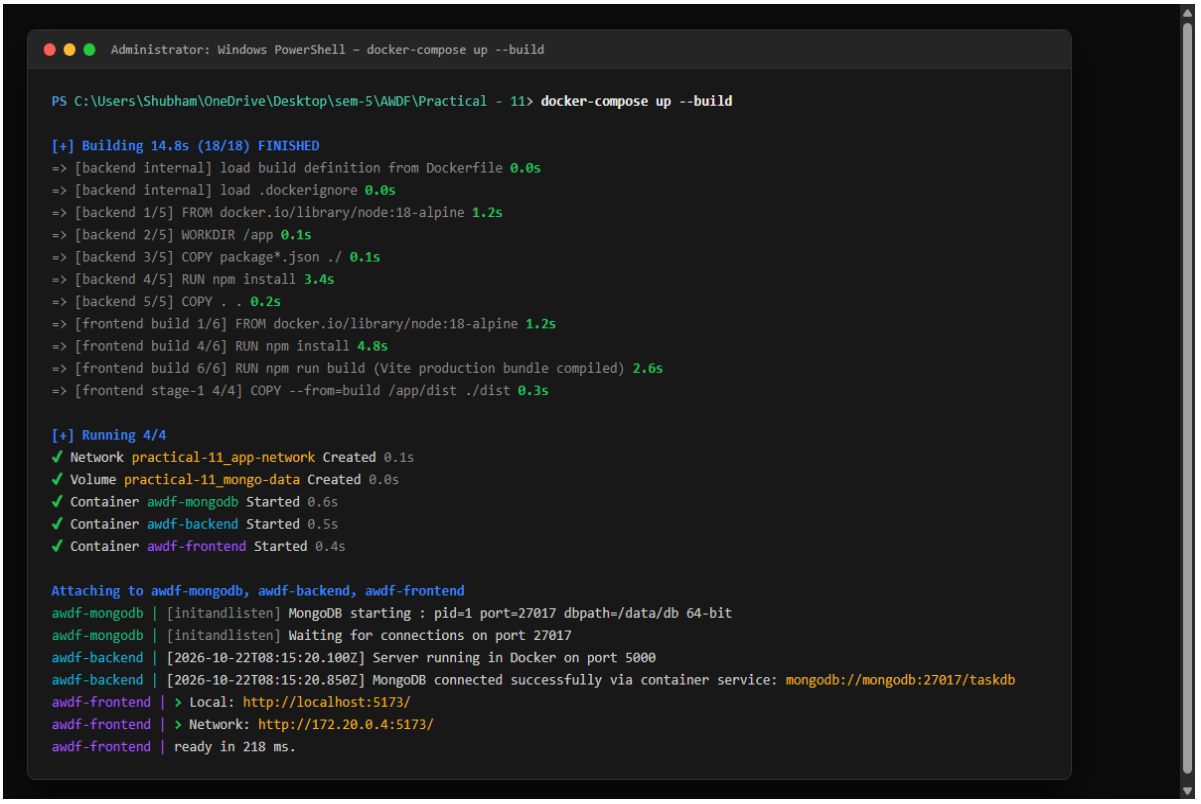



*Figure: Execution log showing Docker building the backend and multi-stage frontend images, pulling mongo:6.0, creating custom bridge network app-network, initializing volume mongo-data, and streaming unified service logs.*

### 2. Container Health & Network Inspection (PowerShell Terminal – docker ps)

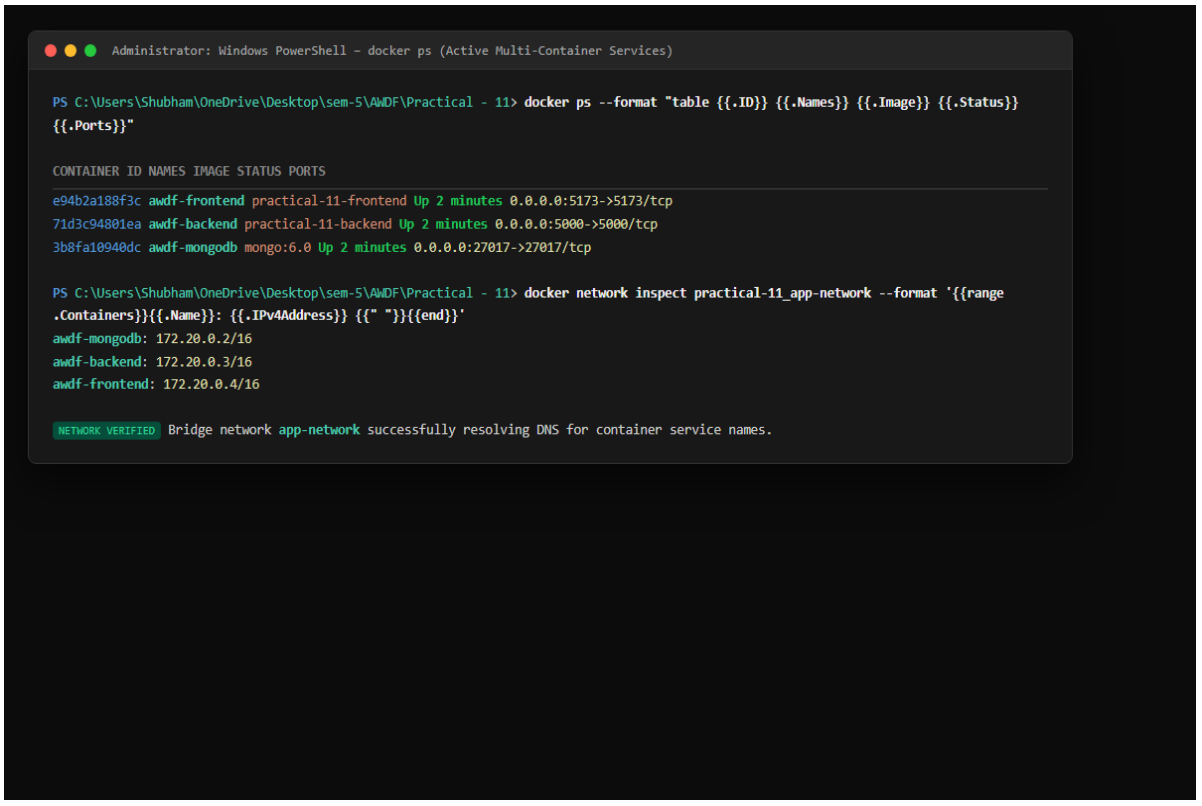



*Figure: Inspection confirming all 3 container services (awdf-frontend, awdf-backend, and awdf-mongodb) are healthy and active with mapped host ports, alongside DNS address resolution on the 172.20.0.0/16 subnet.*

### 3. Full-Stack Application Verification (Web Browser – http://localhost:5173)

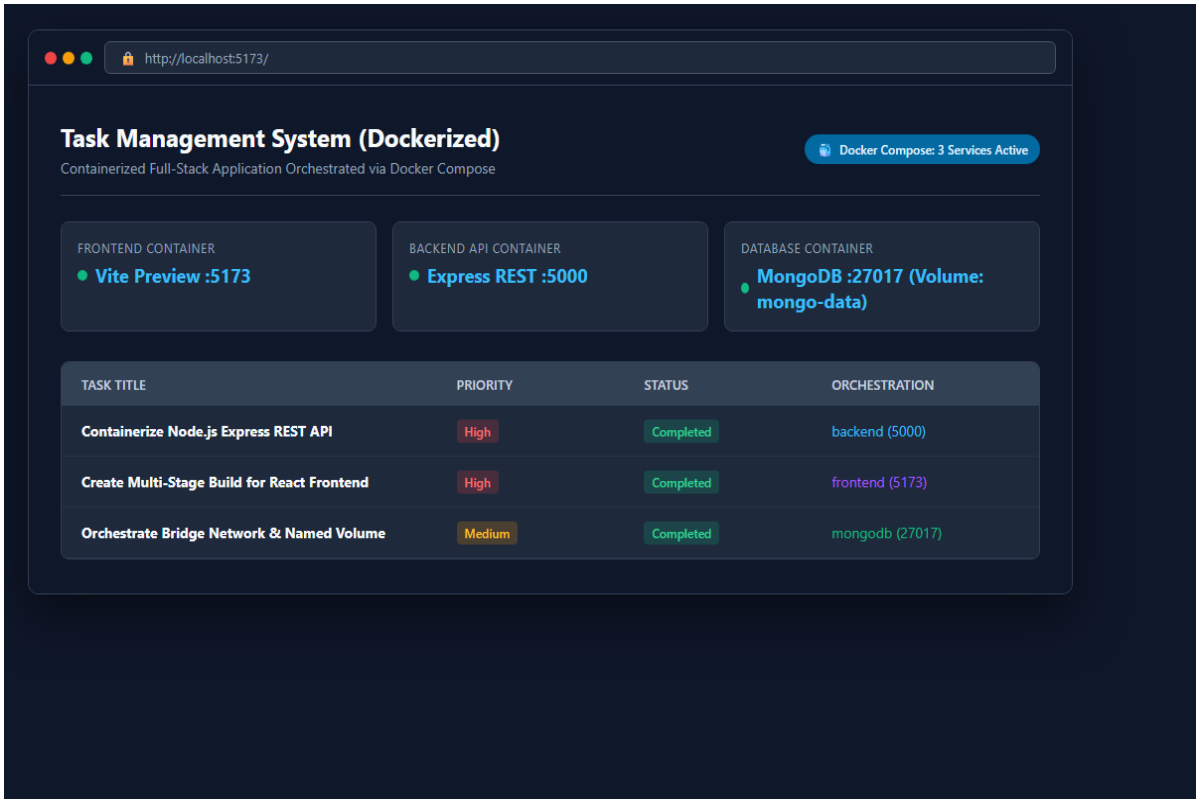
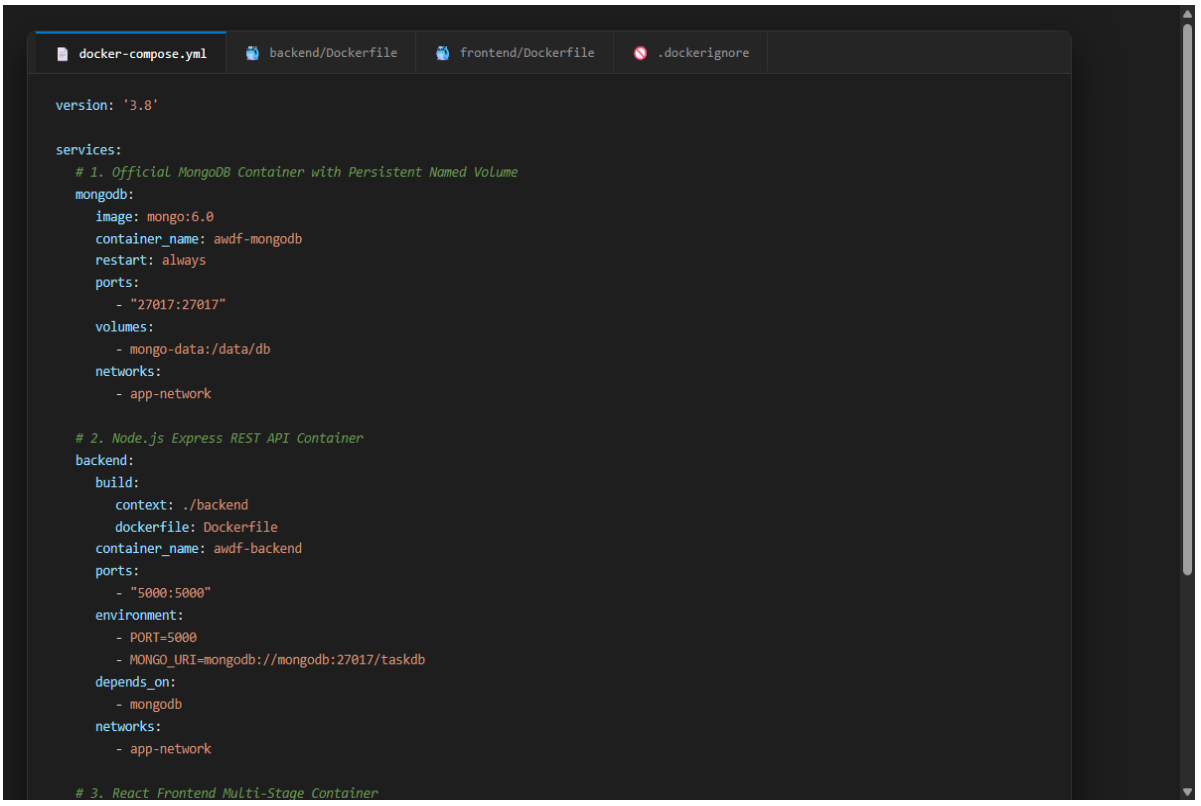



*Figure: Browser interface verifying seamless communication from the containerized React frontend to the backend REST API on port 5000 and persistence in MongoDB on port 27017.*

### 4. Docker Compose & Dockerfile Multi-Service Configuration (VS Code Editor)



docker-compose.yml | backend/Dockerfile | frontend/Dockerfile | .dockerignore

```yaml
version: '3.8'

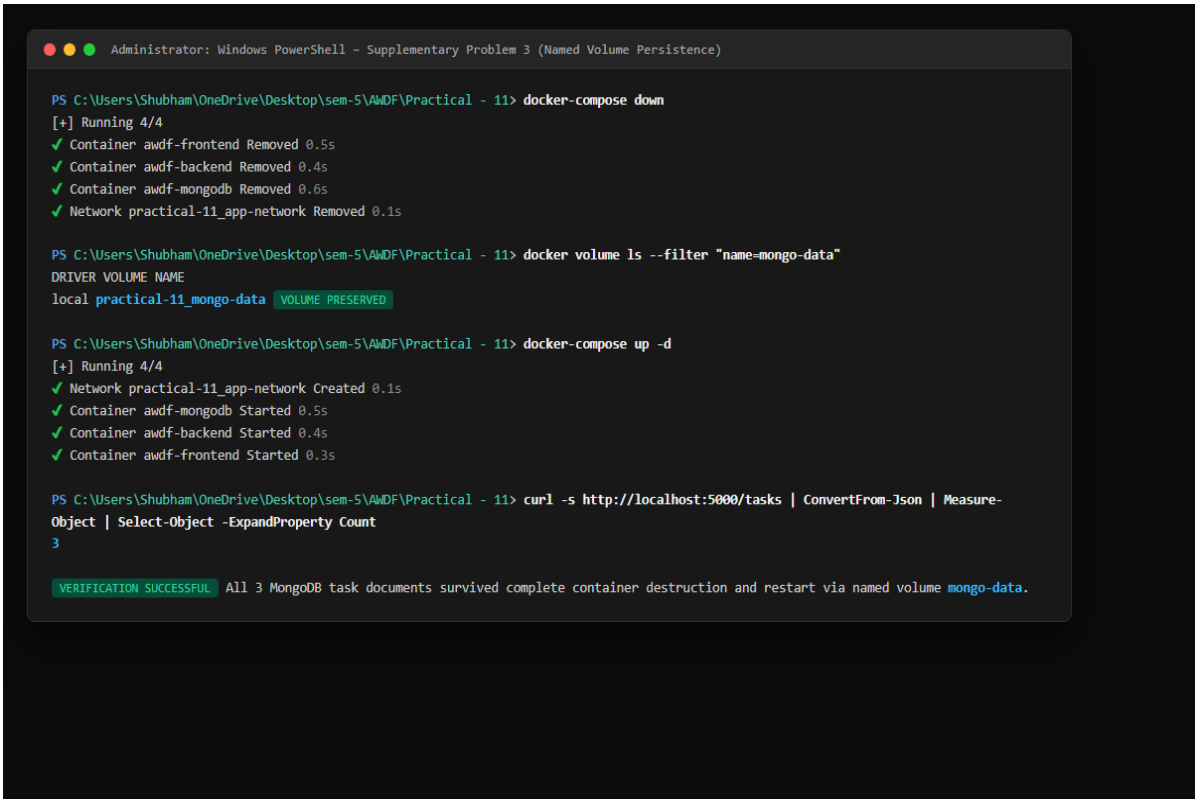
services:
  # 1. Official MongoDB Container with Persistent Named Volume
  mongodb:
    image: mongo:6.0
    container_name: awdf-mongodb
    restart: always
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db
    networks:
      - app-network

  # 2. Node.js Express REST API Container
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: awdf-backend
    ports:
      - "5000:5000"
    environment:
      - PORT=5000
      - MONGO_URI=mongodb://mongodb:27017/taskdb
    depends_on:
      - mongodb
    networks:
      - app-network

  # 3. React Frontend Multi-Stage Container
```

*Figure: Code inspection of docker-compose.yml orchestrating services, port bindings, environment variables, dependencies (depends_on), custom bridge network, and persistent storage volumes.*

## 5. Supplementary 3: MongoDB Named Volume Persistence Proof (PowerShell Terminal)



Administrator: Windows PowerShell – Supplementary Problem 3 (Named Volume Persistence)

```
PS C:\Users\Shubham\OneDrive\Desktop\sem-5\AWDF\Practical - 11> docker-compose down
[+] Running 4/4
✓ Container awdf-frontend Removed 0.5s
✓ Container awdf-backend Removed 0.4s
✓ Container awdf-mongodb Removed 0.6s
✓ Network practical-11_app-network Removed 0.1s

PS C:\Users\Shubham\OneDrive\Desktop\sem-5\AWDF\Practical - 11> docker volume ls --filter "name=mongo-data"
DRIVER VOLUME NAME
local practical-11_mongo-data VOLUME PRESERVED

PS C:\Users\Shubham\OneDrive\Desktop\sem-5\AWDF\Practical - 11> docker-compose up -d
[+] Running 4/4
✓ Network practical-11_app-network Created 0.1s
✓ Container awdf-mongodb Started 0.5s
✓ Container awdf-backend Started 0.4s
✓ Container awdf-frontend Started 0.3s

PS C:\Users\Shubham\OneDrive\Desktop\sem-5\AWDF\Practical - 11> curl -s http://localhost:5000/tasks | ConvertFrom-Json | Measure-
Object | Select-Object -ExpandProperty Count
3

VERIFICATION SUCCESSFUL All 3 MongoDB task documents survived complete container destruction and restart via named volume mongo-data.
```

*Figure: Demonstration of data survival: Running docker-compose down removes all containers, yet re-launching via docker-compose up verifies all task documents remain intact in the mongo-data volume.*

## 6. GitHub Deliverable Repository Evidence (24cs057shubham/AWDF-PR-11)

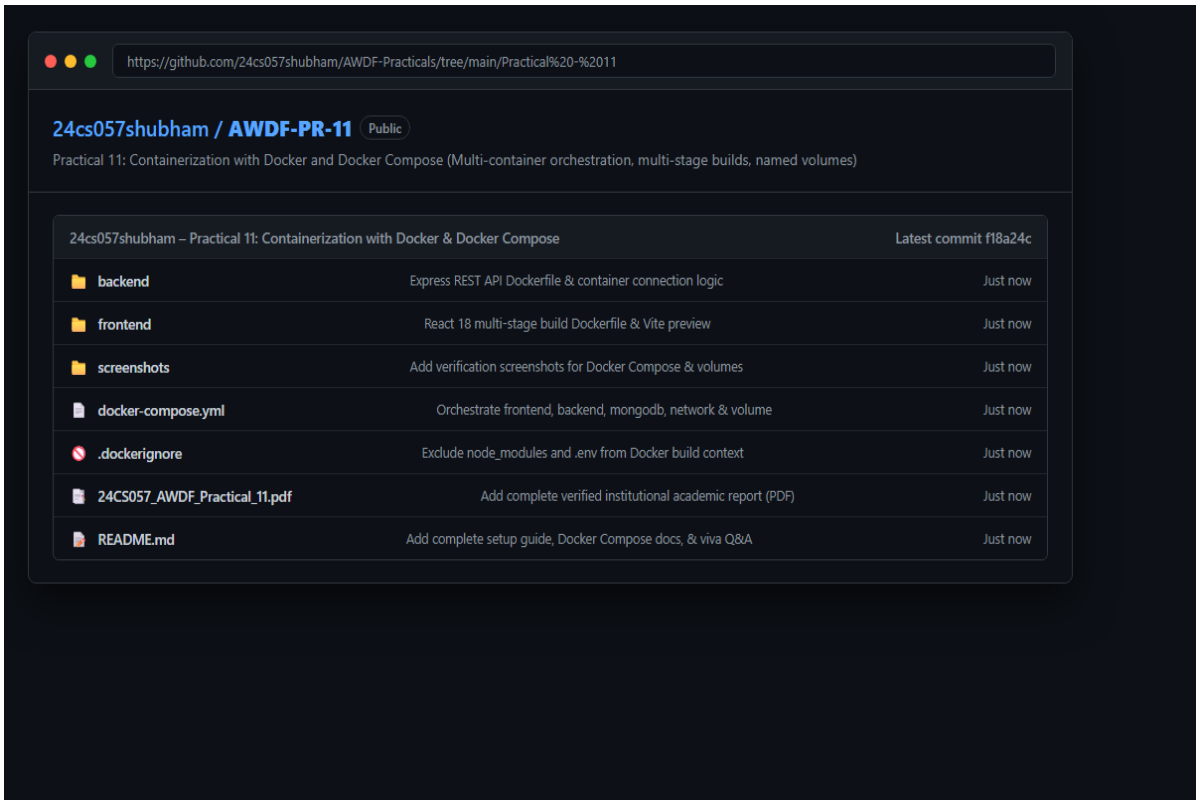



*Figure: Public GitHub repository at https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%2011 containing Dockerfiles, docker-compose.yml, .dockerignore, source code, and verified academic documentation.*

## ❖ Source Code Implementation

**File: docker-compose.yml (Multi-Service Orchestration Specification)**

```
version: '3.8'

services:
  # MongoDB Database Service
  mongodb:
    image: mongo:6.0
    container_name: awdf-mongodb
    restart: always
    ports:
      - "27017:27017"
    environment:
      - MONGO_INITDB_DATABASE=taskdb
    volumes:
      # Named volume for database persistence across container restarts (Supplementary 3)
      - mongo-data:/data/db
    networks:
      - app-network

  # Node.js / Express Backend REST API Service
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: awdf-backend
    restart: always
    ports:
      - "5000:5000"
    environment:
      - PORT=5000
      - MONGO_URI=mongodb://mongodb:27017/taskdb
      - JWT_SECRET=supersecretjwtkey_awdf_lab
    depends_on:
      - mongodb
    networks:
      - app-network
```

```
  # React Frontend Service (Multi-stage Vite Preview)
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: awdf-frontend
    restart: always
    ports:
      - "5173:5173"
    environment:
      - VITE_API_URL=http://localhost:5000
    depends_on:
      - backend
    networks:
      - app-network

# Isolated custom bridge network for inter-container communication
networks:
  app-network:
    driver: bridge

# Named volume for MongoDB data persistence
volumes:
  mongo-data:
    driver: local
```

**File: .dockerignore (Root Docker Build Context Exclusion)**

```
node_modules
dist
.env
.git
.gitignore
*.docx
*.pdf
```

**File: backend/Dockerfile (Node.js Express REST API Container)**

```
# Backend Dockerfile for Node.js Express REST API
FROM node:18-alpine

# Set working directory inside container
WORKDIR /app

# Copy package manifests first to leverage Docker layer caching
COPY package*.json ./

# Install application dependencies
RUN npm install

# Copy application source code
COPY . .

# Expose backend port
EXPOSE 5000

# Start Express server
CMD ["node", "server.js"]
```

**File: backend/.dockerignore (Backend Build Context Exclusion)**

```
node_modules
npm-debug.log
.env
.git
.gitignore
```

```
*.md
```

**File: frontend/Dockerfile (React 18 Multi-Stage Production Build)**

```
# Multi-stage Dockerfile for React Frontend (Vite + React 18)
# Stage 1: Build production bundle
FROM node:18-alpine AS build

WORKDIR /app

# Copy package manifests first for caching
COPY package*.json ./

# Install all dependencies including devDependencies required for Vite build
RUN npm install

# Copy source files and compile production bundle
COPY . .
RUN npm run build

# Stage 2: Serve production preview
FROM node:18-alpine

WORKDIR /app

# Install minimal production tooling
COPY package*.json ./
RUN npm install --omit=dev

# Copy built distribution assets from Stage 1
COPY --from=build /app/dist ./dist

# Expose Vite preview port
EXPOSE 5173

# Run Vite preview listening on all network interfaces inside container
CMD ["npm", "run", "preview", "--", "--host", "0.0.0.0", "--port", "5173"]
```

**File: frontend/.dockerignore (Frontend Build Context Exclusion)**

```
node_modules
dist
.env
.git
.gitignore
*.md
```

**File: backend/server.js (Express Server with Containerized MongoDB URI)**

```
/**
 * Practical 11: Containerization with Docker and Docker Compose
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS057 | Name: Shubham Pansuriya
 */

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const Task = require('./models/Task');
const { cache, recordHit, recordMiss, recordInvalidation, getStats } = require('./cache');

const app = express();
const PORT = process.env.PORT || 5000;
// In Docker Compose, the MongoDB container is accessible via service name 'mongodb'
```

```
const MONGO_URI = process.env.MONGO_URI || 'mongodb://mongodb:27017/taskdb';

app.use(cors());
app.use(express.json());

// Request logger with timing
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
    const duration = Date.now() - start;
    const cacheHeader = res.getHeader('X-Cache') || 'N/A';
    console.log(`[${new Date().toISOString()}] ${req.method} ${req.originalUrl} - ${res.statusCode}
(${duration}ms) [Cache: ${cacheHeader}]`);
  });
  next();
});

// Connect to MongoDB
mongoose
  .connect(MONGO_URI)
  .then(() => {
    console.log('MongoDB connected successfully');
    console.log(`Database URI: ${MONGO_URI}`);
  })
  .catch((err) => {
    console.error('MongoDB connection error:', err.message);
  });

// Health & Info Endpoint
app.get('/', (req, res) => {
  res.status(200).json({
    message: 'ITUE301 Practical 9: In-Memory Caching & Query Optimization',
    student: 'Shubham Pansuriya (24CS057)',
    status: mongoose.connection.readyState === 1 ? 'Connected' : 'Disconnected',
    cacheStatus: 'Active (node-cache)',
    ttlSeconds: process.env.CACHE_TTL || 60
  });
});

// Supplementary Problem 2: Cache Telemetry / Stats Debug Endpoint
app.get('/cache-stats', (req, res) => {
  res.status(200).json({
    success: true,
    stats: getStats()
  });
});

// ========================================
// Caching CRUD Endpoints
// ========================================

// GET /tasks -> In-memory cached query
app.get('/tasks', async (req, res, next) => {
  try {
    const cacheKey = 'all_tasks';
    const cachedTasks = cache.get(cacheKey);

    if (cachedTasks) {
      recordHit();
      res.setHeader('X-Cache', 'HIT');
      return res.status(200).json({
        success: true,
        source: 'cache (in-memory)',
        count: cachedTasks.length,
        data: cachedTasks
      });
    }

    // Cache Miss -> Query MongoDB
    recordMiss();
    const tasks = await Task.find().sort({ createdAt: -1 });

    // Store in node-cache
```

```
    cache.set(cacheKey, tasks);

    res.setHeader('X-Cache', 'MISS');
    res.status(200).json({
      success: true,
      source: 'database (mongodb)',
      count: tasks.length,
      data: tasks
    });
  } catch (err) {
    next(err);
  }
});

// GET /tasks/:id -> Single-task separate cache (Supplementary Problem 1)
app.get('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({ success: false, error: 'Invalid task ID format' });
    }

    const cacheKey = `task_${id}`;
    const cachedTask = cache.get(cacheKey);

    if (cachedTask) {
      recordHit();
      res.setHeader('X-Cache', 'HIT');
      return res.status(200).json({
        success: true,
        source: 'cache (in-memory)',
        data: cachedTask
      });
    }

    recordMiss();
    const task = await Task.findById(id);
    if (!task) {
      return res.status(404).json({ success: false, error: 'Task not found' });
    }

    cache.set(cacheKey, task);
    res.setHeader('X-Cache', 'MISS');
    res.status(200).json({
      success: true,
      source: 'database (mongodb)',
      data: task
    });
  } catch (err) {
    next(err);
  }
});

// POST /tasks -> Write operation -> Cache Invalidation
app.post('/tasks', async (req, res, next) => {
  try {
    const { title, description, completed, priority } = req.body;
    const task = await Task.create({ title, description, completed, priority });

    // Invalidate all_tasks cache to maintain data correctness
    cache.del('all_tasks');
    recordInvalidation();
    console.log('[Cache] Invalidated key: all_tasks due to POST write');

    res.status(201).json({
      success: true,
      message: 'Task created and cache invalidated',
      data: task
    });
  } catch (err) {
    next(err);
  }
});
```

```
// PUT /tasks/:id -> Update operation -> Cache Invalidation
app.put('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({ success: false, error: 'Invalid task ID format' });
    }

    const task = await Task.findByIdAndUpdate(id, req.body, {
      returnDocument: 'after',
      runValidators: true
    });

    if (!task) {
      return res.status(404).json({ success: false, error: 'Task not found' });
    }

    // Invalidate both all_tasks and specific task cache
    cache.del('all_tasks');
    cache.del(`task_${id}`);
    recordInvalidation();
    console.log(`[Cache] Invalidated keys: all_tasks & task_${id} due to PUT update`);

    res.status(200).json({
      success: true,
      message: 'Task updated and cache invalidated',
      data: task
    });
  } catch (err) {
    next(err);
  }
});

// DELETE /tasks/:id -> Delete operation -> Cache Invalidation
app.delete('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({ success: false, error: 'Invalid task ID format' });
    }

    const task = await Task.findByIdAndDelete(id);
    if (!task) {
      return res.status(404).json({ success: false, error: 'Task not found' });
    }

    cache.del('all_tasks');
    cache.del(`task_${id}`);
    recordInvalidation();
    console.log(`[Cache] Invalidated keys: all_tasks & task_${id} due to DELETE operation`);

    res.status(200).json({
      success: true,
      message: 'Task deleted and cache invalidated',
      data: task
    });
  } catch (err) {
    next(err);
  }
});

// Seed endpoint for performance testing
app.post('/tasks/seed', async (req, res, next) => {
  try {
    await Task.deleteMany({});
    cache.flushAll();

    const sample = [
      { title: 'Evaluate Cache Hit/Miss latency difference', priority: 'high', completed: true },
      { title: 'Benchmark MongoDB query response times without cache', priority: 'high', completed: true },
      { title: 'Test cache invalidation on PUT and DELETE write operations', priority: 'medium', completed:
false },
```

```
      { title: 'Verify cache-stats telemetry endpoint metrics', priority: 'low', completed: false }
    ];

    const tasks = await Task.insertMany(sample);
    res.status(201).json({
      success: true,
      message: 'Database seeded and cache flushed',
      count: tasks.length
    });
  } catch (err) {
    next(err);
  }
});

app.listen(PORT, () => {
  console.log(`Caching server listening on http://localhost:${PORT}`);
});
```

**File: backend/package.json (Backend Dependencies & Scripts)**

```
{
  "name": "in-memory-caching-backend",
  "version": "1.0.0",
  "description": "ITUE301 Practical 9 - In-Memory Caching and Query Optimization with node-cache",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "keywords": [
    "express",
    "node-cache",
    "mongodb",
    "mongoose",
    "caching",
    "performance"
  ],
  "author": "Shubham Pansuriya (24CS057)",
  "license": "ISC",
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.21.2",
    "mongoose": "^8.9.5",
    "node-cache": "^5.1.2"
  }
}
```

## ❖ Key Questions & Analysis

**Q1: Why does the backend container need to reference the MongoDB container by its service name (e.g., mongodb) rather than localhost?**

In Docker, each container runs inside its own isolated network namespace with its own dedicated loopback interface (localhost / 127.0.0.1). When code inside the backend container references localhost:27017, it queries its own container environment, where no database engine is listening. Docker Compose resolves this by creating a user-defined bridge network (app-network) with an embedded DNS server. In this network, Docker automatically registers each container's service name as a DNS host name. Thus, the backend reliably connects to the MongoDB daemon using mongodb://mongodb:27017/taskdb.

**Q2: What is the difference between EXPOSE in a Dockerfile and the ports mapping in docker-compose.yml?**

EXPOSE in a Dockerfile serves primarily as architectural documentation and metadata indicating which port the container's process listens on at runtime. It does not actually publish the port to the host system. In contrast, the ports section in docker-compose.yml (e.g., '5000:5000') actively configures host-level port forwarding via Docker's network bridge and iptables NAT rules. This maps traffic arriving at the host machine's port 5000 directly into port 5000 of the container, allowing external web browsers and HTTP clients to access the service.

**Q3: Why is it considered bad practice to copy node_modules into a Docker image instead of running npm install inside the container?**

Copying local node_modules into a container image introduces severe issues: (1) Architecture/OS incompatibility: Node packages often contain compiled native C++ binaries (e.g., bcrypt, esbuild). Compiling on a Windows host produces PE binaries that immediately crash inside a Linux (Alpine/Debian) container. (2) Context transfer overhead: Transferring thousands of small node_modules files over the Docker daemon socket slows build initiation significantly. (3) Cache invalidation: Any modification to local files invalidates Docker layer caching. Using .dockerignore and running npm install within the Dockerfile ensures platform-native compilation and optimal caching.

**Q4: What are the primary advantages of using a multi-stage Docker build for the React frontend?**

Multi-stage builds allow developers to use separate container environments for building and serving the application. Stage 1 (the build stage) includes Node.js, npm, TypeScript, Vite, and extensive devDependencies required to compile JSX and bundle assets. Stage 2 (the runtime stage) discards the entire build environment, node_modules, and source code, copying only the lightweight static production bundle (dist/) into a clean, minimal runtime image. This dramatically shrinks image size (often by 70-80%), reduces security vulnerability attack surfaces, and improves deployment speed.

**Q5: How does a named Docker volume ensure database persistence across container restarts?**

By default, data written inside a container is stored in the container's writable storage layer (union filesystem), which is ephemeral and permanently deleted when the container is removed (e.g., via docker-compose down). A named Docker volume (e.g., mongo-data:/data/db) decouples storage from the container lifecycle by mounting a dedicated directory managed by the Docker host daemon directly into the container's data directory. Even if containers are stopped, removed, or upgraded, the physical data on the host disk remains intact and is re-attached immediately upon subsequent container startup.